Aufgabe 3: Nasser Hund

Team-ID: 00563

Team-Name: Matthieu Dang

Bearbeiter dieser Aufgabe:
Matthieu Dang

Inhaltsverzeichnis

1 Lösungsidee **1**
- 1.1 Kleinste Strecke zwischen zwei Strecken . . . 2
- 1.2 Distanz zwischen zwei Punkte . . . 2
- 1.3 Kleinste Distanz zwischen Strecke und Punkt . . . 2
- 1.4 Berechnung des Lotes . . . 2
- 1.5 Bildung der Funktion der Strecke . . . 2

2 Umsetzung **3**
- 2.1 Klasse für Strecken . . . 3
- 2.2 Ausnahme: Strecke senkrecht . . . 3
- 2.3 Schnittpunkt von Funktionen . . . 3
- 2.4 Berechnung des Lotfußpunktes . . . 3
- 2.5 Prüfung, ob ein Punkt auf einer Strecke liegt . . . 3
- 2.6 Distanz vom Lotfußpunkt und einem Endpunkt . . . 3
- 2.7 Kleinste Distanz zwischen zwei Strecken . . . 4
- 2.8 Ausführung des Programmes . . . 4

3 Werkzeuge **4**

4 Beispiele **4**

5 Quellcode **4**
- 5.1 Klassen für Punkte und Stecken . . . 4
- 5.2 Funktion für Schnittpunkte . . . 5
- 5.3 Distanz im Lot . . . 5
- 5.4 kleinste Distanz von zwei Linien . . . 6
- 5.5 kleinste Distanz insgesamt . . . 6

1 Lösungsidee

In dieser Aufgabe wird eine Datei mit Koordinaten von Endpunkten von Wegen und Eckpunkte von Seen eingegeben. Dabei muss die größte mögliche Leine für einen Hund berechnet werden, wenn man sich auf die Wege bewegt, sodass der Hund nicht ans Wasser kann. Diese Länge der Leine ist also gleichzeitig die größte Distanz zwischen einem Ufer und einem Weg.

1.1 Kleinste Strecke zwischen zwei Strecken

Um diese kleinste Distanz zwischen allen Wegen und Ufern zu berechnen, braucht man eine Weise, die kleinste Distanz zwischen zwei Strecken zu berechnen, die dann auf jedem Weg mit jedem Ufer anwendet.

Die kleinste Strecke zwischen zwei Strecken endet immer an einem Endpunkt von einer Strecke, wenn diese sich nicht kreuzen. Wege und Ufer können sich nicht kreuzen, weshalb das ignoriert werden kann. Dies kann so begründet werden, dass diese Strecken zu Geraden gemacht werden, die bei dem Schnittpunkt aufhören. es entsteht eine V-Figur mit beiden Strecken als Schenkel. Jede beliebige Strecke zwischen dem Weg und Ufer kann immer weiter zu dem Schnittpunkt bewegt werden und wird immer kleiner dabei. Die Verschiebung kann man machen, bis eine Strecke endet, damit die Strecke noch vom Weg zum Ufer liegt.

Die kleinste Strecke ist also eine Strecke von einer Strecke und einem Endpunkt. Bei der Berechnung der kleinsten Strecke wird für jede Strecke der kleinste Abstand zu dem anderen Punkt berechnet. Es kann aber auch sein, dass diese kleinste Strecke zwischen zwei Endpunkten liegt, Es wird also auch die Distanz zwischen allen Endpunkten von beiden Strecken berechnet.

1.2 Distanz zwischen zwei Punkte

Die Distanz zwischen zwei Punkten berechnet man mit dem Satz des Pythagoras ($c = \sqrt{a^2 + b^2}$) , a und b sind jeweils die Differenzen der x- und y-Koordinaten der Punkte.

1.3 Kleinste Distanz zwischen Strecke und Punkt

Die kleinste Verbindung zwischen einer Geraden und einem Punkt ist immer das Lot. Die kürzeste Strecke ist also der Lotfußpunkt und der Punkt, der Lotfußpunkt ist der Schnittpunkt vom Lot und der Strecke. Falls der Lotpunkt also auf der Strecke ist, die auf der Geraden liegt, geht, kann man davon die kleinste Distanz berechnen. Man braucht also eine Weise, das Lot berechnen zu können.

1.4 Berechnung des Lotes

Das Lot auf einer Strecke durch einen Punkt gefällt ist in Wirklichkeit nur eine Senkrechte durch einen Punkt. Die Senkrechte einer linearen Funktion lässt sich einfach berechnen, weshalb ich die Strecken als Funktionen darstelle. Lineare Funktionen werden mit $f(x) = mx + t$ dargestellt, sie sind Geraden mit der Steigung m und dem y-Achsenabschnitt t. Die Senkrechte zu einer Linearen Funktion besitzt immer die Steigung $-\frac{1}{m}$. Die ganze Funktion ist also

$$f_{\text{senk}}(x) = f(x) \perp p = -\frac{1}{m}x + t_{\text{senk}}$$

Es wird anschließend der y-Achsenabschnitt berechnet, damit diese Gerade durch den Punkt geht, und das Lot wird. Dafür setzt man die y- und x-Koordinate des Punktes in die Funktion ein und löst nach t auf.

$$y_p = -\frac{1}{m}x_p + t \rightarrow t = y_p - (-\frac{1}{m})x_p$$

Den Lotfußpunkt erhält man, wenn man anschließend den Schnittpunkt des Lotes und der Geraden der Strecke nimmt. Man kann damit anschließend die Distanz zwischen dem Punkt und der Strecke berechnen, wenn sich der Lotpunkt im von der Strecke definierten Intervall befindet.
Der Schnittpunkt von zwei Funktionen *f(x)* und *g(x)* lässt sich so berechnen:

$$m_1x + t_1 = m_2x + t_2 \rightarrow x = \frac{t_1 - t_2}{m_2 - m_1}$$

Man setzt dann den x-Wert in eine der Funktionen ein, um den ganzen Punkt zu erhalten. Falls die Steigungen gleich sind, gibt es keinen Schnittpunkt, da die Geraden parallel verlaufen.

1.5 Bildung der Funktion der Strecke

Um den Schnittpunkt vom Lot und der Strecke und so den Lotfußpunkt zu berechnen, benötigt man, dass die Strecke als Funktion dargestellt ist. Die Form $f(x) = mx + t$ kennen wir schon. Man ermittelt m, indem man $\frac{\Delta y}{\Delta x}$ berechnet. Anschließend löst man die Funktion nach um, und setzt einen der Enden ein, um t zu erhalten.

2 Umsetzung

2.1 Klasse für Strecken

Um mit der Umsetzung zu beginnen, habe ich die Ufer und Wege als Objekte der Klasse Line dargestellt. Diese besitzt erstmal zwei Punkte, also beide Enden der Strecke. Diese Punkte sind jeweils Teil der Klasse Point, die nur eine x- und y-Koordinate besitzen. Sobald die Strecken aber erstellt werden, werden diese direkt als Funktionen dargestellt. Es wird also als erstes die Steigung *self.steepness* mit $\frac{\Delta y}{\Delta x}$ der beiden Enden berechnet. Anschließend kommt man auf den y-Achsenabschnitt *self.y_ intercept* durch das Einsetzen eines Endpunktes in $t = y_p - mx_p$. Zuletzt berechnet man die Steigung der Senkrechten zu aller Strecken mit $-\frac{1}{m}$.

2.2 Ausnahme: Strecke senkrecht

Falls eine Strecke senkrecht ist, stellt es sich heraus, dass diese keine Steigung besitzen kann, da $\frac{\Delta y}{\Delta x}$ immer 0 im Nenner besitzt. Deshalb, wenn beide x-Koordinaten der Endpunkte der Strecke gleich sind, habe ich mich entschieden, trotzdem einen Wert in *self.steepness* einzusetzen um diese zu kennzeichnen. Dieser sollte math.inf, also unendlich sein, das ist auch sehr praktisch, denn man kann immer noch math.inf mit einer anderen Zahl oder sich selbst vergleichen kann. Die Funktion benötigt aber auch einen y-Achsenabschnitt. Dieser existiert in fast allen Fällen nicht, da die Gerade parallel zur y-Achse verläuft. Damit die Funktion aber eindeutig ist, wird in *self.y_ intercept* anstelle des y-Achsenabschnittes die x-Koordinate gespeichert, an der sich die Funktion entlang bewegt, also die gemeinsame x-Koordinate der Endpunkte. Die Steigung der Senkrechten ist dann immer 0. Ist die Steigung der Stecke selbst 0, so besitzt die Senkrechte selber die Steigung math.inf.

2.3 Schnittpunkt von Funktionen

Die Funktion *intersection()* nimmt alle Parameter von zwei linearen Funktionen ein, und berechnet daraus den Schnittpunkt. Das ist nur möglich, wenn beide Geraden nicht parallel sind, also dieselbe Steigung besitzen. Falls das jedoch der Fall ist, wird None zurückgegeben, sonst wird ein Punkt ausgegeben. Wenn eine der Strecken senkrecht ist und die Steigung math.inf, ist die x-Koordinate des Schnittpunktes immer der y-Achsenabschnitt dieser, da die Strecke sich nur an eine x-Koordinte befindet.

2.4 Berechnung des Lotfußpunktes

Die Funktion *intersection_ perpendicular_ line_ point()* berechnet den Schnittpunkt von einer Senkrechten, also dem Lot durch einen gegebenen Punkt. Man muss dafür erst den y-Achsenabschnitt der Funktion des Lotes bestimmen. Da man die ganze Funktion für *intersection()* braucht. Man benutzt wieder $t = y_p - mx_p$, in m setzt man die Senkrechtensteigung der Strecke ein, die schon berechnet wurde. Für eine Steigung von math.inf wird wieder die x-Koordinate des Punktes als y-Achsenabschnitt genutzt. Nachdem man die ganze Funktion bestimmt hat, kann man diese in *intersection()* einsetzen, um den Schnittpunkt und so den Lotfußpunkt zu finden.

2.5 Prüfung, ob ein Punkt auf einer Strecke liegt

Um zu prüfen ob ein Punkt auf einer Strecke liegt, muss man nur prüfen, ob die x-Koordinate des Punktes in der Funktion der Strecke die y-Koordinate des Punktes ergibt. Außerdem muss der Punkt sich im von den Endpunkten abgegrenzten Intervall befinden. Bei einer senkrechten, muss man, nur prüfen, ob der Punkt, dieselbe x-Koordinate, wie die Strecke besitzt und zwischen beiden Endunkten auf der y-Achse liegt.

2.6 Distanz vom Lotfußpunkt und einem Endpunkt

Um die Distanz vom Lotfußpunkt, braucht man die Koordinaten des Lotfußpunktes, den man mit *intersection_ perpendicular_ line_ point()* berechnet hat. Falls dieser auf der Strecke liegt, ist diese Distanz gültig, da es dann eine Distanz zum Weg bzw. Ufer ist. Es wird von den beiden Punkten der Abstand anschließend mit dem Satz des Pythagoras berechnet.

2.7 Kleinste Distanz zwischen zwei Strecken

Um die kleinste Distanz zwischen zwei Strecken zu berechnen gibt es zuletzt die Funktion *smallest_ distance()*. Darin wird von jedem Endpunk von einer Strecke die Distanz zu jedem anderen Endpunkt mit dem Satz des Pythagoras berechnet. Anschließend wird zu jeder Strecke die Distanz im Lot zu den anderen Endpunkten berechnet. Der kleinste all dieser Werte, ist dann die kleinste Distanz zwischen beiden Strecken.

2.8 Ausführung des Programmes

Die Ausführung aus dem Computer-Terminal erfolgt so, dass man sich erst im Verzeichnis des Programms Aufgabe3.py befinden muss. Das Programm wird durch den Befehl *python Aufgabe3.py eingabedatei.txt* gestartet. *eingabedatei.txt* wird dabei mit dem Pfad der Datei ersetzt, in der sich die Koordinaten der Eckpunkte der Seen und Endpunkte der Wege befinden. Man sollte aber beachten, dass der Python-Interpreter im PATH sein soll.

Bei der Ausführung werden diese Koordinaten zu Strecken der Klasse Line übersetzt. Jeder See wird kreisförmig zusammengeknüpft, sodass man immer eine Strecke von einem Eckpunkt zum nächsten bildet. Die letzten und ersten Punkte werden auch zusammen zu einer Strecke gemacht, damit der See geschlossen ist. Für jedes Seeufer wird für jeden Weg die kleinste Distanz mit *smallest_ distance()* berechnet. Der kleinste Wert von allen Distanz ist die größte Linie, die dann benutzt werden kann, ohne dass der Hund ins Wasser geht.

3 Werkzeuge

Es wurde keine KI bei der Ideengewinnung oder bei der Umsetzung benutzt, ich habe jedoch die kostenlose ChatGPT-Version ausschließlich zur Überprüfung des Codes auf Leichtsinnsfehler benutzt. Bei Unklarheiten, wie etwas in Python implementiert wird, wie z.B. bei der Unendlichkeit mit math.inf, habe ich Hilfe bei StackOverflow gefunden.[1].

Um die Terminalargumente zu implementieren habe ich die Library sys in python importiert. *sys.argv[1]* nimmt dann die nächste Terminaleingabe nach der Ausführungsdatei, die dann in meinem Code geöffnet wird.

Außerdem habe ich die math-library in Python benutzt, die viele mathematische Funktionen und Konstanten beinhält. math.inf repräsentiert einen Wert, der unendlich groß ist.

4 Beispiele

Die Ausgaben meines Programmes zu den Beispielen der BWINF-Webeite:

- **hund01.txt** : 4.036036763977875
- **hund02.txt**: 4.2098784926737425
- **hund03.txt**: 2.23606797749979
- **hund04.txt:** 1.6712580435934707
- **hund05.txt:** 50.320350312991536
- **hund06.txt**: 199.5725023626933

5 Quellcode

5.1 Klassen für Punkte und Stecken

```
class Point:

    def __init__(self, x, y):
        self.x = x
        self.y = y
```

[1] https://stackoverflow.com/questions/7781260/how-to-represent-an-infinite-number-in-python[13.11.2025]

```
class Line:

    def __init__(self, p0, p1):
        self.p0 = p0
        self.p1 = p1

        # findet Parameter m; t fuer Gerade durch Strecke: f(x) = mx + t

        # Spezialfall: dx = 0 > linie senkrecht, Steigung als math.inf
        if p0.x == p1.x:
            self.steepness = math.inf
        else:
            self.steepness = (p0.y - p1.y) / (p0.x - p1.x) # berechnung der Steigung durch dx/dy

        # Lineare Funktion: f(y) = mx + t; y_achsenabschnitt auf engl. y-intercept

        if self.steepness == math.inf:
            # linie senkrecht > schneidet nie y_achse, y_intercept als x-koordinate wo die Strecke ist
            self.y_intercept = p0.x
        else:
            self.y_intercept = p0.y - (p0.x * self.steepness) # y1 = m * x1 + t -> t = y1 - (m * x1)

        # Findet Steigung der Senkrechten, immer senkrecht zur Tangente

        if self.steepness == math.inf:
            self.perp_steepness = 0.0 # senkrecht zur y-achse ist paralell zur x-achse
        elif self.steepness == 0.0:
            self.perp_steepness = math.inf # senkrecht zur x-achse ist paralell zur y-achse
        else:
            self.perp_steepness = -1 / self.steepness
```

5.2 Funktion für Schnittpunkte

```
def intersection(line0_steepness, line0_y_intercept, line1_steepness, line1_y_intercept):
    """Gibt x und y-koordinate des Schnittpunktes zurueck, falls keiner vorhanden: return None"""

    if  line0_steepness == line1_steepness: # linien paralell
        return None

    elif line0_steepness == math.inf:
    # line0 ist senkrecht -> Schnittpunkt bei x = line0_y_intercept (siehe Zeile 27)
        return Point(line0_y_intercept, (line1_steepness * line0_y_intercept) + line1_y_intercept)
        # line0_y_intercept in Funktion von line1 einsetzen

    elif line1_steepness == math.inf:
        return Point(line1_y_intercept, (line0_steepness * line1_y_intercept) + line0_y_intercept)
        # line1.y_intercept in Funktion von line0 einsetzen

    else:
        intersection_x =  (line0_y_intercept - line1_y_intercept) / (line1_steepness - line0_steepness)
        # m1*x + t1 = m2*x +t2 -> x = (t1 - t2) / (m2 - m1)
        intersection_y = line0_steepness * intersection_x + line0_y_intercept
        # x_koordinate in Funktion von line0 einsetzen
        return Point(intersection_x, intersection_y)
```

5.3 Distanz im Lot

```
def intersection_perpendicular_line_point(line, point):
    """Berechnet Schnittpunkt vom Lot gefaellt durch point auf line und line"""

    # y-Achsenabschnitt der Senkrechten berechnen um in intersection() enizusetzen
    if line.perp_steepness == math.inf:
        perp_y_intercept = point.x # (siehe Zeile 27)
    else:
        perp_y_intercept = point.y - (point.x * line.perp_steepness)
        # y = m * x + t -> t = y - (m * x)
```

```
10
    # in intersection() einsetzen
12  return intersection(line.steepness, line.y_intercept, line.perp_steepness, perp_y_intercept)

14 def is_on_line(line, point):
    if line.steepness == math.inf:
16      # punkt muss auf x_koordinate von line, also line.y_intercept sein, und zwischen
        #hoechsten und tiefsten Punkt von line sein
18      return line.y_intercept == point.x and min(line.p0.y, line.p1.y) <= point.y <=
            max(line.p0.y, line.p1.y)
20  else:
        # punkt muss auf Geraden sein und im von den Endpunkten abgegrenzten Bereich sein
22      return point.y == point.x * line.steepness + line.y_intercept and min(line.p0.x, line.p1.x) <=
            point.x <= max(line.p0.x, line.p1.x)
24
def distance_line_perpendicular_line(line, point):
26  """Distanz zischen point und Schnittpunkt vom Lot gefaellt durch point auf line und line"""
    other_point = intersection_perpendicular_line_point(line, point) # Lotfusspunkt
28
    if other_point is None:
30      return None
    elif not is_on_line(line, other_point):
32      return None # distanz nicht gueltig
    else:
34      return distance(point, other_point)
```

5.4 kleinste Distanz von zwei Linien

```
def smallest_distance(line0, line1):
2   """Gibt kleinste Distanz zwischen zwei Strecken zurueck"""

4   distances = []

6   # alle Distanzen zwischen endpunkte der Linien
    for line0_endpoint in [line0.p1, line0.p0]:
8       for line1_endpoint in [line1.p1, line1.p0]:
            distances.append(distance(line0_endpoint, line1_endpoint))
10
    # alle Distanzen zwischen Endpunkt und anderer Linie
12  # mit distance_line_perpendicular_line() berechnet
    for line0_endpoint in [line0.p1, line0.p0]:
14      distance_line_point_perpendicular = distance_line_perpendicular_line(line1, line0_endpoint)
        if distance_line_point_perpendicular is not None:
16          distances.append(distance_line_point_perpendicular)
    for line1_endpoint in [line1.p1, line1.p0]:
18      distance_line_point_perpendicular = distance_line_perpendicular_line(line0, line1_endpoint)
        if distance_line_point_perpendicular is not None:
20          distances.append(distance_line_point_perpendicular)
```

5.5 kleinste Distanz insgesamt

```
shortest_leash_possible = math.inf
2
    for path in paths:
4       for lakeshore in lakeshores:
            shortest_leash_possible = min(shortest_leash_possible, smallest_distance(path, lakeshore))
6
    print(shortest_leash_possible)
```